

Rich Text System

▼ Relevant source files

Frontend/STUDER/src/app/app.routes.ts

Frontend/STUDER/src/app/shared/components/rich-text/entity-popover/entity-popover.component.css

Frontend/STUDER/src/app/shared/components/rich-text/entity-popover/entity-popover.component.html

Frontend/STUDER/src/app/shared/components/rich-text/entity-popover/entity-popover.component.ts

Frontend/STUDER/src/app/shared/components/rich-text/manual-entity/manual-entity.component.css

Frontend/STUDER/src/app/shared/components/rich-text/manual-entity/manual-entity.component.html

Frontend/STUDER/src/app/shared/components/rich-text/manual-entity/manual-entity.component.ts

Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.css

Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.html

Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.ts

Frontend/STUDER/src/app/shared/rich-text.index.ts

Frontend/STUDER/src/app/shared/services/entity-cache.service.ts

Frontend/STUDER/src/app/shared/services/rich-text-parser.service.ts

Frontend/STUDER/src/styles.css

The **Rich Text System** is a shared frontend subsystem responsible for parsing and rendering interactive text content. It identifies specific entity patterns (mentions, tags, courses, etc.) within raw strings and transforms them into interactive components with hoverable previews (popovers) and routing capabilities.

Architecture Overview

The system follows a pipeline where raw text is tokenized by a service and then rendered by a component that manages the lifecycle of entity-specific UI elements.

System Data Flow

The following diagram illustrates the transformation from a raw string to a rendered UI with interactive popovers.

Rich Text Transformation Pipeline

Sources:	Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.ts	20-33
	Frontend/STUDER/src/app/shared/components/rich-text/entity-popover/entity-popover.component.ts	48-89

Tokenization and Parsing

The `RichTextParserService` uses regular expressions to split text into an array of `TextToken` objects. Each token is either a plain string or a recognized entity.

Supported Entities

Entity Type	Prefix	Regex Pattern / Logic	Color Variable
User	@	@username	--entity-user-color
Tag	#	#tagname	--entity-tag-color
Course	&	&course_id	--entity-course-color
Contest	\$	\$contest_id	--entity-contest-color
Block	%	%block_id	--entity-block-color

Sources:	Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.ts	46-54
	Frontend/STUDER/src/styles.css	108-114

Core Components

`RichTextComponent` (`studer-rich-text`)

The primary entry point for rendering rich text. It receives a `text` input and uses `RichTextParserService` to generate the view.

- **Parsing Logic:** Triggered on `ngOnInit` and `ngOnChanges`
Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.ts 31-40
- **Routing:** Dynamically generates internal routes based on entity type (e.g., `/user/@username`, `/tags/tagname`)
Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.ts 46-55
- **Interaction:** Emits `entityClick` events when a link is clicked, allowing parent components to override default routing
Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.ts 57-63

ManualEntityComponent (`studer-manual-entity`)

Used when an entity is already known and doesn't need parsing from a string. It wraps content in the same styles and popover logic as `RichTextComponent`

Frontend/STUDER/src/app/shared/components/rich-text/manual-entity/manual-entity.component.ts 19-22

EntityPopoverComponent (`studer-entity-popover`)

A floating overlay that displays metadata about an entity.

- **Data Fetching:** For `user` types, it calls `UserService.getByUsername` if the data is not in the cache
Frontend/STUDER/src/app/shared/components/rich-text/entity-popover/entity-popover.component.ts 61-86
- **Mocking:** For other types (`tag`, `course`, `contest`, `block`), it currently generates mock data and stores it in the `EntityCacheService`
Frontend/STUDER/src/app/shared/components/rich-text/entity-popover/entity-popover.component.ts 90-118
- **Positioning:** Uses `fixed` positioning calculated from the trigger element's `getBoundingClientRect()`
Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.ts 65-75

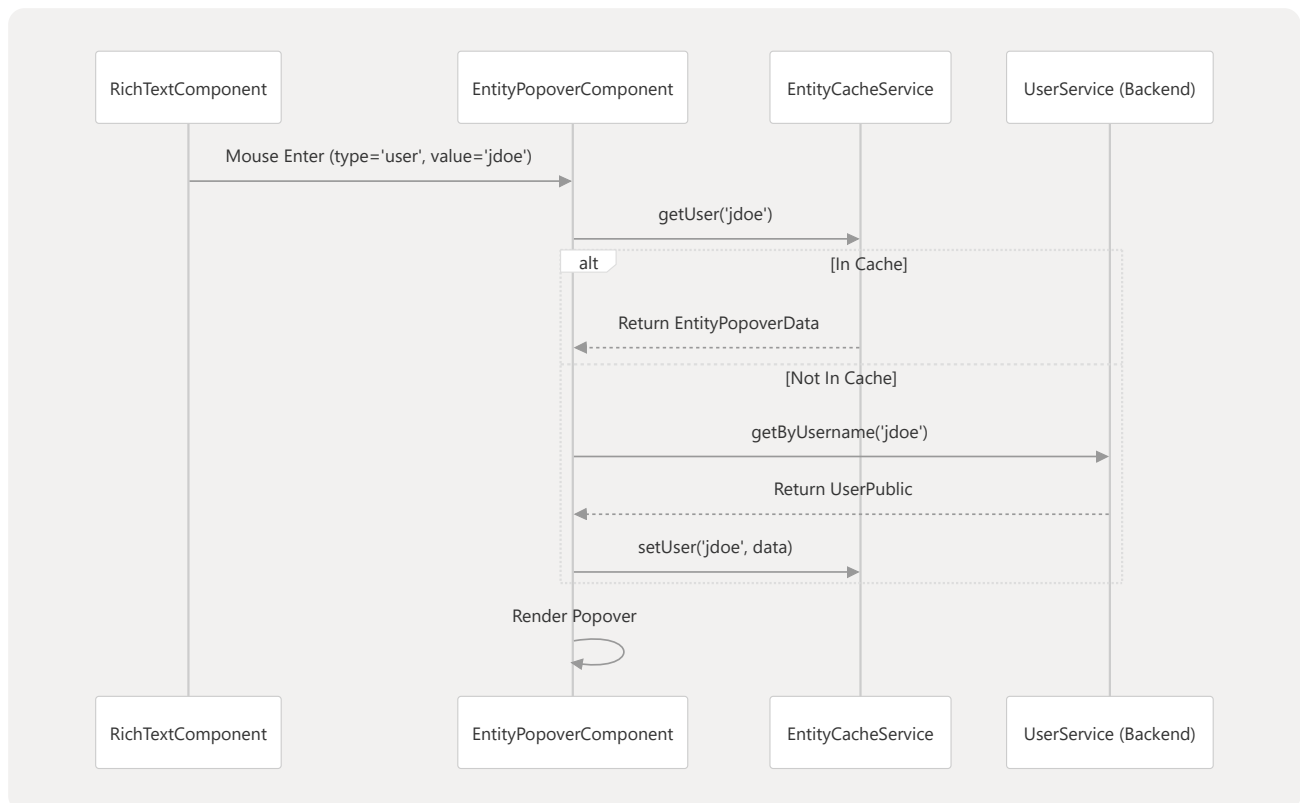
Sources:

Frontend/STUDER/src/app/shared/components/rich-text/entity-popover/entity-popover.component.ts 1-173

Data Persistence and Deduplication

The `EntityCacheService` prevents redundant API calls when the same entity appears multiple times or is hovered repeatedly.

Entity Resolution Sequence



Sources:

Frontend/STUDER/src/app/shared/components/rich-text/entity-popover/entity-popover.component.ts

57-89

Frontend/STUDER/src/app/shared/rich-text.index.ts 4

Styling and Theming

The system uses CSS variables defined in `styles.css` to ensure consistent entity coloring across light and dark themes.

- **Base Class:** `.entity` handles transitions, font weight, and hover underlines

Frontend/STUDER/src/styles.css 177-190

- **Type Modifiers:** Classes like `.entity--user` or `.entity--tag` apply specific colors using variables like `--entity-user-color`

Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.css 27-45

- **Popover Styling:** The `.entity-popover` uses `var(--color-bg)` and `var(--overlay-border)` to adapt to the active theme

Frontend/STUDER/src/app/shared/components/rich-text/entity-popover/entity-popover.component.css 1-11

Sources: Frontend/STUDER/src/styles.css 108-114

Frontend/STUDER/src/app/shared/components/rich-text/rich-text.component.css 12-45

Public API (Barrel Export)

All rich text utilities are exported via `rich-text.index.ts` to simplify imports across the feature modules.

- **Services:** `RichTextParserService`, `EntityCacheService`.
- **Components:** `RichTextComponent`, `EntityPopoverComponent`, `ManualEntityComponent`.
- **Interfaces:** `TextToken`, `EntityClickEvent`, `ManualEntityClickEvent`.

Sources: Frontend/STUDER/src/app/shared/rich-text.index.ts 1-13